studocu

Midterm assess cs4231 2022

Parallel and Distributed Algorithms (National University of Singapore)

Scan to open on Studocu

Studocu is not sponsored or endorsed by any college or university
Downloaded by ?? ? (kjaehyeok21@gmail.com)

INSTRUCTIONS

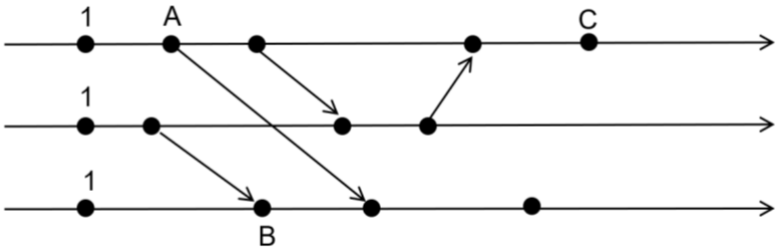
1. This assessment contains **NINE** problems and comprises **FIVE** pages, including this page.
2. You have total **90 minutes** to work on this assessment.
3. Answer **ALL** questions within the space provided in this booklet.
4. This is a **Closed Book** assessment.
5. This assessment should be completed by each student individually, without help from anyone else or from the Internet.
6. When answering the questions, you can invoke lemmas/theorems that we have already covered in class, unless the question prohibits this. When you do so, you should first state the lemmas/theorems that you wish to invoke, and then explain how you want to invoke it in your context. You CANNOT invoke lemmas/theorems/results from anywhere else (i.e., those not covered in class). Answering the questions will not require you to draw any diagram. But you still can draw diagram in Word if you prefer. You are NOT allowed to insert a photo or scanned image/text into the Microsoft word file. You can use standard C/Java mathematical operators. For example, "*" means multiplication, and "^" means exponentiation. You can also define additional functions. For example, you can define the function sqrt(x) to return the square root of x.
7. At the end of the assessment, you should name the Word file using your student number (e.g., "A0123456X.doc") and then upload the Word file to Luminus. The submission MUST be in Word format. You are NOT allowed to upload a photo or scanned copy of the Word document --- such submissions will get zero mark. You should submit once and ONLY ONCE. If you want to update, you should delete your old submission and then submit a new one.
8. Write your student number below. Do not write your name.

STUDENT NO: _____

Problem 1. (0 marks)
Have you double-checked that you have typed your student number correctly in the above? (yes/no)
_____

Downloaded by ?? ? (kjaehyeok21@gmail.com)

Problem 2. (6 marks)
Consider the following execution of a certain distributed system. Imagine that we maintain logical clocks in the system so that each event gets a logical clock value. The logical clock value of the first event on each process is already indicated in the figure.



The logical clock value of the event A should be _____. (You only need to write a number. No explanation is needed.)

The logical clock value of the event B should be _____. (You only need to write a number. No explanation is needed.)

The logical clock value of the event C should be _____. (You only need to write a number. No explanation is needed.)

Problem 3. (4 marks)
We have two processes, P1 and P2, in a parallel system. There are two shared integer variables, X and Y, and their initial values are both 0. Following is the pseudo-code for P1 and P2:

P1's pseudo-code (executes only once)	*P2's pseudo-code (executes only once)*
X = 3+X;	Y = 2+X;
X = X-Y;	Y = 5*Y;

Please give an interleaving execution of P1 and P2 in the following table, so that at the end of the execution, the value of X is -7 and the value of Y is 10. You should also indicate the value of X and Y after each step.

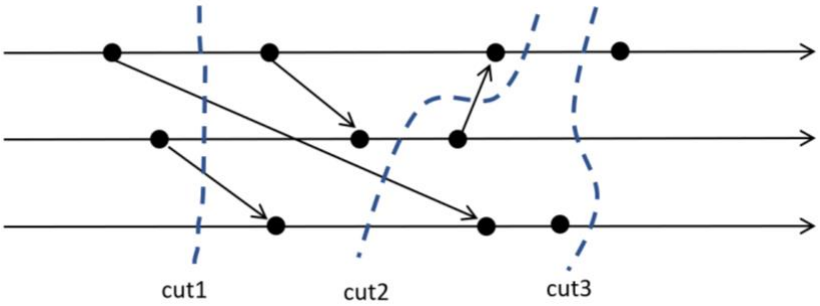
P1	*P2*	*value of X after each step*	*value of Y after each step*
		-7	**10**

Please give another interleaving execution of P1 and P2 in the following table, so that at the end of the execution, the value of X is -22 and the value of Y is 25. You should indicate value of X and Y after each step.

P1	*P2*	*value of X after each step*	*value of Y after each step*
		-22	**25**

Downloaded by ?? ? (kjaehyeok21@gmail.com)

Problem 4. (6 marks)



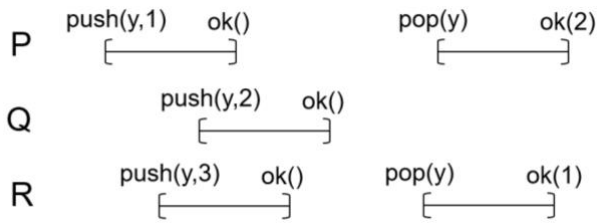
Consider the above execution of a certain distributed system.

Does cut1 correspond to a consistent global snapshot? Indicate "yes" or "no" (no explanation needed) _____

Does cut2 correspond to a consistent global snapshot? Indicate "yes" or "no" (no explanation needed) _____

Does cut3 correspond to a consistent global snapshot? Indicate "yes" or "no" (no explanation needed) _____

Problem 5. (4 marks)



Assume that y is a shared stack that is initially empty.

Is the above history sequentially consistent? Indicate "yes" or "no" (no explanation needed) _____

Is the above history linearizable? Indicate "yes" or "no" (no explanation needed) _____

Downloaded by ?? ? (kjaehyeok21@gmail.com)

Problem 6. (6 marks)
Alice has designed the following mutual exclusion algorithm for *n* processes, using Java-style monitor:

```
flag is a Boolean object, that is initialized to false and is shared among all processes

RequestCS() {
        while (flag == true) { synchronized (flag) { flag.wait(); } }
        flag = true;
}

ReleaseCS() {
        flag = false;
        synchronized (flag) { flag.notifyAll(); }
}
```

Alice only requires the "Mutual Exclusion" property and the "Progress" property, and she does not need "No Starvation". Explain clearly why Alice's algorithm fails to achieve her goal (you only need to show that either "Mutual Exclusion" or "Progress" is violated --- you do not need to show that both are violated):

Explain clearly how to modify Alice's algorithm to make it correct (i.e., to achieve both "Mutual Exclusion" and "Progress"):

Problem 7. (4 marks)
We explained in class that a history H is *linearizable* if there exists some history S such that i) S is equivalent to H, ii) S is a legal sequential history, and iii) S preserves the external order in H. We now call such an S as a *supporter* for H. A linearizable history H may have more than one supporter. For a linearizable history H containing total *n* operations, what is the maximum number of supporters that H can possibly have?

The maximum possible number of supporters is _____.

Justify your answer by constructing an H with so many supporters:

Downloaded by ?? ? (kjaehyeok21@gmail.com)

Problem 8. (4 marks)
Consider the vector clock protocol in the context of a distributed system with n processes. For 1< = i <= n, we already know that the first event on process i is a local processing event, and that event has a vector clock timestamp whose i-th entry is 1 and whose remaining entries are all 0. Besides these n events (i.e., the first event on each process), there are k additional events in the execution (these k events may be on the same process, or may be on different processes). This means there are total n+k events in this execution. Each such event will also have a vector clock timestamp. Define the *weight* of an event to be the sum of all the n entries in its vector clock timestamp. Define the *weight* of this execution to be the maximum weight among the n+k events, and let x denote the weight of this execution. We assume that both n and k are even numbers, and that k is much larger than n.

What is the maximum possible value for x (as a function of n and k)? _____

Please justify your answer by constructing an execution with such an x value:

Problem 9. (4 points)
Consider the following algorithm for mutual exclusion between 2 processes:

Initially, "wantCS[0]" = false, "wantCS[1]" = false, and "turn" = 0.

Protocol for P0	Protocol for P1

// invoke RequestCS to enter critical section
void RequestCS() {
  wantCS[0] = true;
  while (wantCS[1]) {
  if (turn == 1) {
  wantCS[0] = false;
  while (turn == 0) { ;}
  wantCS[0] = true;
  }
  }
}
 	
// invoke ReleaseCS to exit critical section
void ReleaseCS() {
  turn = 1;
  wantCS[0] = false;
}
	// invoke RequestCS to enter critical section
	void RequestCS() {
	  wantCS[1] = true;
	  while (wantCS[0]) {
	  if (turn == 0) {
	  wantCS[1] = false;
	  while (turn == 1) { ;}
	  wantCS[1] = true;
	  }
	  }
	}
	
	// invoke ReleaseCS to exit critical section
	void ReleaseCS() {
	  turn = 0;
	  wantCS[1] = false;
	}

Do you feel that the above algorithm ensures **no starvation**? If yes, provide **a rigorous proof**. If not, construct a counter-example. Your counter-example needs to clearly indicate how the executions of different processes are interleaved, in order for starvation to occur. In particular, you need to **indicate, at each step, which process executes which statement**.

Do you think the above algorithm correctly ensures no starvation? (yes/no) _____
Give your proof or counter-example below (only indicating yes/no but without giving a proof/counter-example will earn you ZERO mark):

END-OF-ASSESSMENT

Downloaded by ?? ? (kjaehyeok21@gmail.com)